

Kiểm thử phần mềm

Khoa CNTT
ĐH Đông Á

Bài 7. Kiểm thử tự động

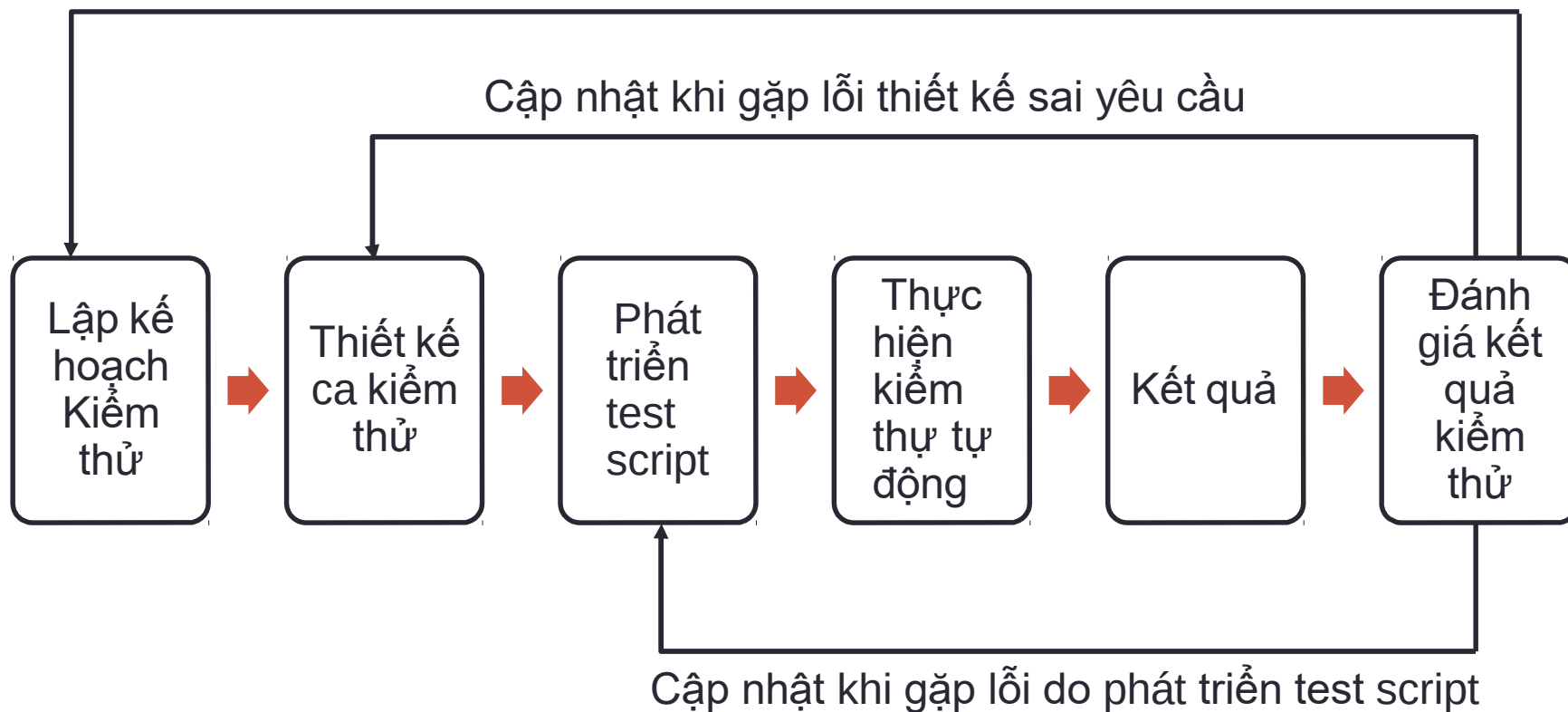
1. Tổng quan về kiểm thử tự động
2. Quy trình kiểm thử tự động
3. Ưu, nhược điểm của kiểm thử tự động
4. Công cụ tự động QTP, Junit, Selenium, Robotium...

1. Tổng quan kiểm thử tự động

- Kiểm thử tự động: áp dụng các công cụ giúp thực hiện việc kiểm thử phần mềm.
- Nên sử dụng công cụ tự động khi:
 - Không đủ tài nguyên
 - Kiểm thử hồi quy
 - Kiểm tra khả năng vận hành của phần mềm trong môi trường đặc biệt.
- Test script: nhóm mã lệnh đặc tả kịch bản dùng để tự động hóa một trình tự kiểm thử.
- Test script: có thể tạo thủ công hoặc tạo tự động dùng công cụ kiểm thử tự động

2. Quy trình kiểm thử tự động

Cập nhật khi kiểm thử chưa thỏa mức độ bao phủ yêu cầu phần mềm



2. Quy trình kiểm thử tự động

1. *Tạo test script*

- Giai đoạn này ta dùng test tool để ghi lại các thao tác lên PM cần kiểm tra và tự động sinh ra test script

2. *Chỉnh sửa test script*

- Chỉnh sửa lại test script thực hiện kiểm tra theo đúng yêu cầu đặt ra, cụ thể là làm theo test case cần thực hiện

3. *Chạy test script để kiểm thử tự động*

- Giám sát hoạt động kiểm tra phần mềm của test script

4. *Đánh giá kết quả*

- Kiểm tra kết quả thông báo sau khi thực hiện kiểm thử tự động. Sau đó bổ sung, chỉnh sửa những sai sót

3. Ưu, nhược điểm của kiểm thử tự động

- Ưu điểm:

- Kiểm thử phần mềm không cần can thiệp của tester
- Giảm chi phí thực hiện kiểm tra số lượng lớn các test case hoặc test case lặp lại nhiều lần
- Giả lập tình huống khó có thể thực hiện bằng tay

- Nhược điểm:

- Mất chi phí tạo các script để thực hiện kiểm thử tự động
- Tốn chi phí dành cho bảo trì các script
- Đòi hỏi tester phải có kỹ năng tạo và thay đổi script cho phù hợp test case
- Không áp dụng tìm được các lỗi mới cho phần mềm

4. Kiểm thử tự động với QTP, Junit, Selenium, Robotium ...

- <Sinh viên thuyết trình>

Câu hỏi kiểm tra

1. Khái niệm kiểm thử đơn vị. Nêu mối quan hệ giữa kiểm thử đơn vị và tạo lập mã nguồn.
2. Tại sao kiểm thử dữ liệu qua giao diện lại có đặc trưng liên quan đến định dạng và số lượng.
3. Nêu khái niệm cuống và bánh lái. Khi nào cần sử dụng cuống và bánh lái, lấy ví dụ.

Câu hỏi kiểm tra

- Khái niệm phần mềm, chất lượng phần mềm
- Trình bày về lỗi, sai sót, hỏng hóc, phân tích và minh họa các nguyên nhân gây ra lỗi
- Trình bày về quy trình kiểm thử, các sản phẩm của quy trình kiểm thử
- Trình bày sự khác biệt của kiểm thử hộp đen và hộp trắng, lấy các ví dụ minh họa

Bài tập 1

Chia nhóm, lập trình bài toán giải phương trình $ax^2 + bx + c = 0$

- Khi $a = 0$: Thông báo “Không phải bậc 2”
- Khi $\Delta < 0$: Thông báo “Vô nghiệm”
- Khi $\Delta = 0$: Tính và thông báo giá trị nghiệm kép
- Khi $\Delta > 0$: Tính và thông báo giá trị 2 nghiệm
- Hãy xây dựng bảng quyết định liệt kê toàn bộ các điều kiện và hành động
- Viết các test case từ bảng quyết định đó
- Thực thi tự động các testcase trên bằng công cụ phù hợp. Ví dụ sử dụng JUnit nếu lập trình bằng java

4. CÔNG CỤ JUNIT

JUnit:

Công cụ kiểm thử cho các đơn vị chương trình viết bằng Java

Một khi bộ kiểm thử được thiết lập, nó có thể tự động chạy mỗi khi mã thay đổi.

JUnit khuyến khích các nhà phát triển viết các kịch bản kiểm thử, chèn các mã kiểm thử vào mã nguồn Java và thực hiện chúng để phát hiện các lỗi bên trong đơn vị chương trình.

CÔNG CỤ JUNIT

JUnit:

Khác với các công cụ khác, JUnit không hỗ trợ cơ chế sinh các ca kiểm thử.

Hiện nay, JUnit đã được tích hợp trong Eclipse và hỗ trợ rất đặc lực cho quá trình kiểm thử.

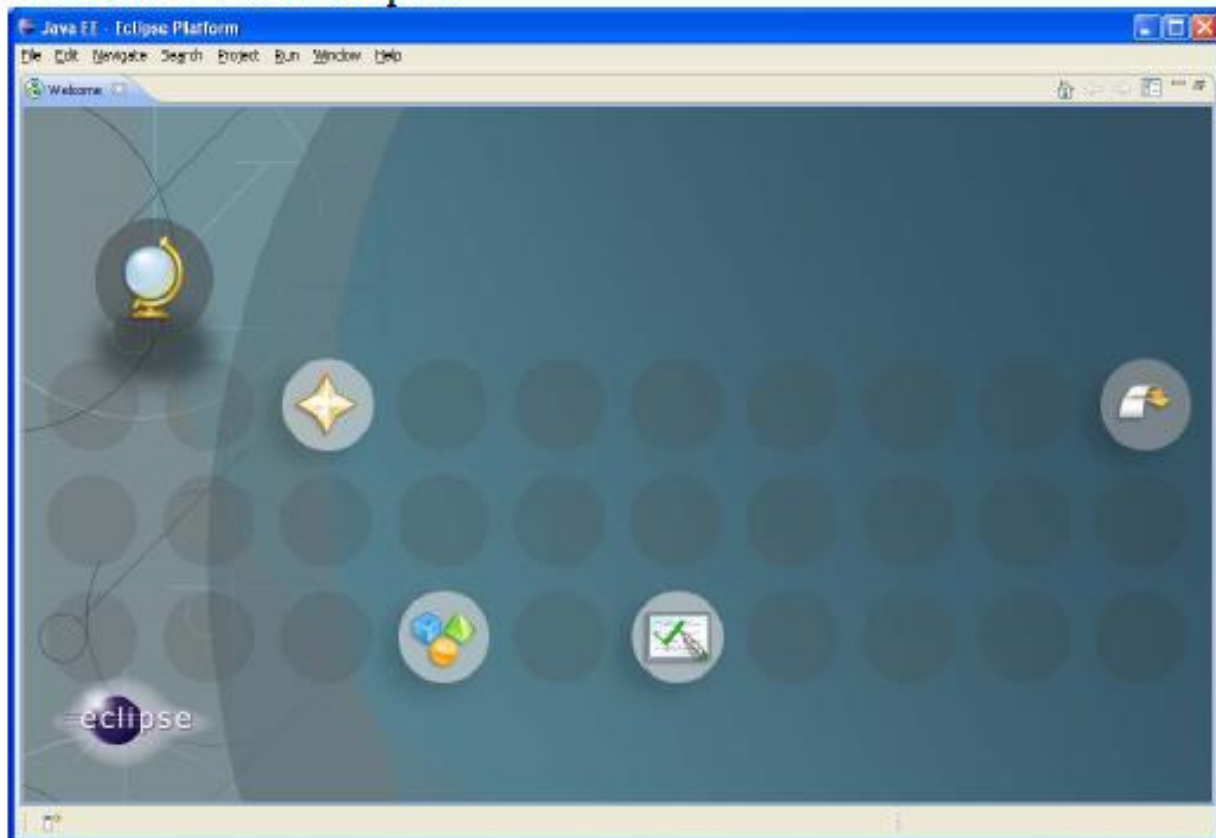
CÔNG CỤ JUNIT

1. Cài đặt Java và Eclipse

- _ Download JDK (phiên bản 1.5 trở lên) tại www.java.sun.com
- _ Cài đặt JDK theo hướng dẫn
- _ Download Eclipse phiên bản Europa (Java Developers hoặc Java EE Developers) tại www.eclipse.org
- _ Giải nén file eclipse.zip ra một thư mục

2. Thay đổi giao diện

Màn hình Welcome của Eclipse:

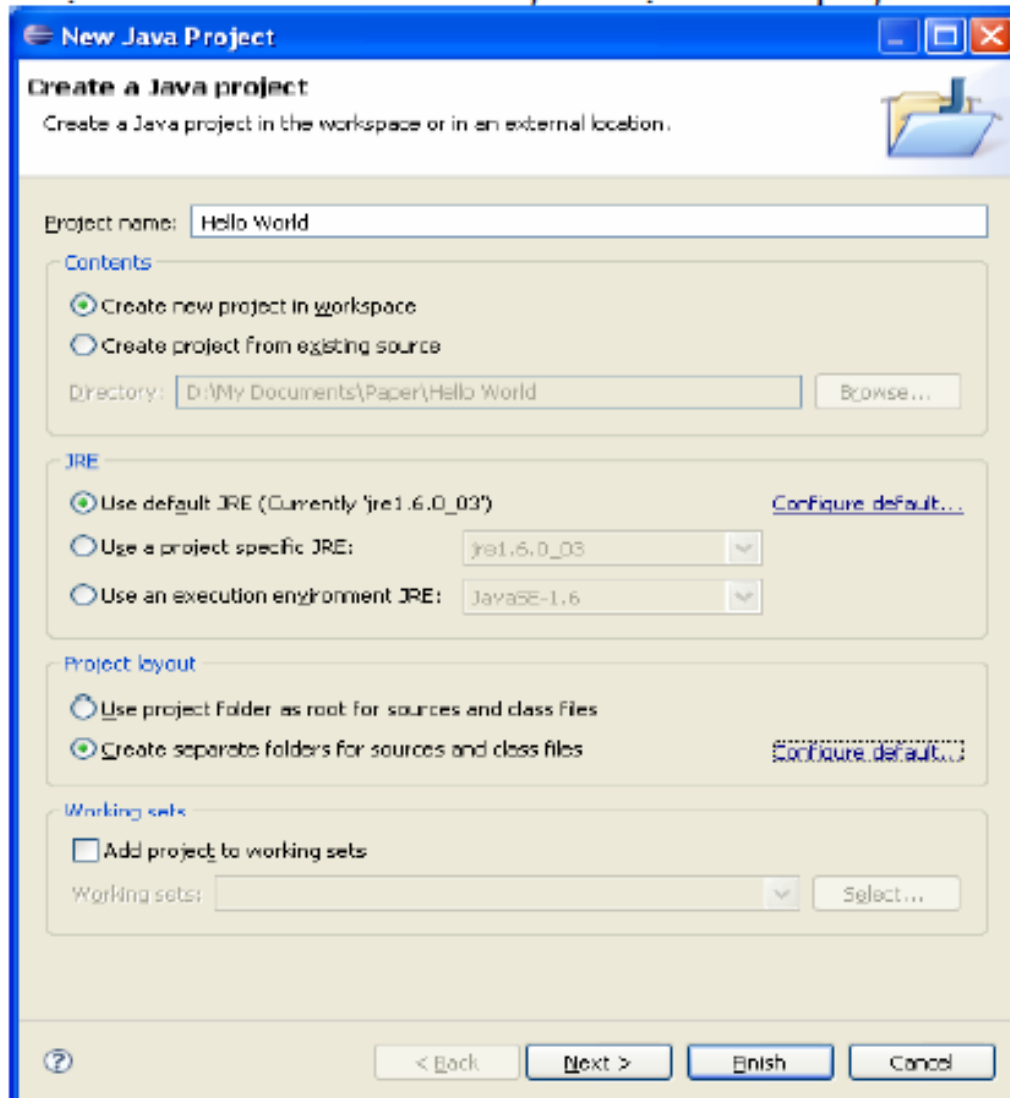


CÔNG CỤ JUNIT

3. Khởi tạo và quản lý Project

a. Tạo một project java

Chọn File → New → Java Project. Đặt tên cho project và chọn Next



The screenshot shows the 'New Java Project' dialog box in the Eclipse IDE. The title bar reads 'New Java Project'. Below the title bar, it says 'Create a Java project' and 'Create a Java project in the workspace or in an external location.' with a folder icon. The 'Project name' field contains 'Hello World'. Under the 'Contents' section, 'Create new project in workspace' is selected. The 'Directory' field shows 'D:\My Documents\Paper\Hello World' with a 'Browse...' button. Under the 'JRE' section, 'Use default JRE (Currently 'jre1.6.0_03')' is selected, with a 'Configure default...' link. Other options are 'Use a project specific JRE:' with a dropdown showing 'jre1.6.0_03' and 'Use an execution environment JRE:' with a dropdown showing 'Javase-1.6'. Under the 'Project layout' section, 'Create separate folders for sources and class files' is selected, with a 'Configure default...' link. Under the 'Working sets' section, 'Add project to working sets' is unchecked. The 'Working sets' field is empty with a 'Select...' button. At the bottom, there are buttons for '< Back', 'Next >', 'Finish', and 'Cancel'.

New Java Project

Create a Java project
Create a Java project in the workspace or in an external location.

Project name: Hello World

Contents

- ☒ Create new project in workspace
- ☐ Create project from existing source

Directory: D:\My Documents\Paper\Hello World Browse...

JRE

- ☒ Use default JRE (Currently 'jre1.6.0_03') [Configure default...](#)
- ☐ Use a project specific JRE: jre1.6.0_03
- ☐ Use an execution environment JRE: Javase-1.6

Project layout

- ☐ Use project folder as root for sources and class files
- ☒ Create separate folders for sources and class files [Configure default...](#)

Working sets

- ☐ Add project to working sets

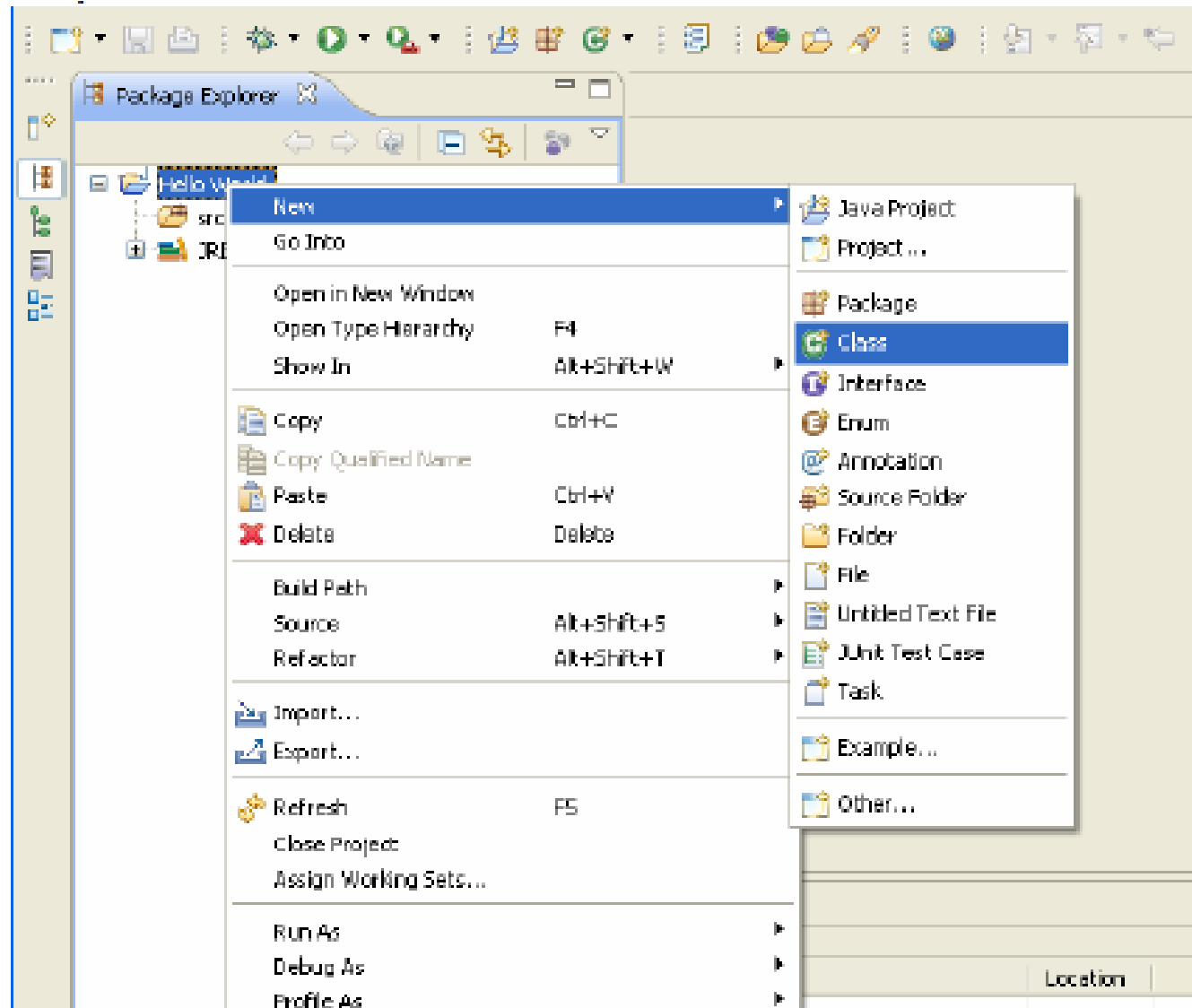
Working sets: Select...

< Back Next > Finish Cancel

CÔNG CỤ JUNIT

b. Thêm một class mới vào project

Chọn New → Class

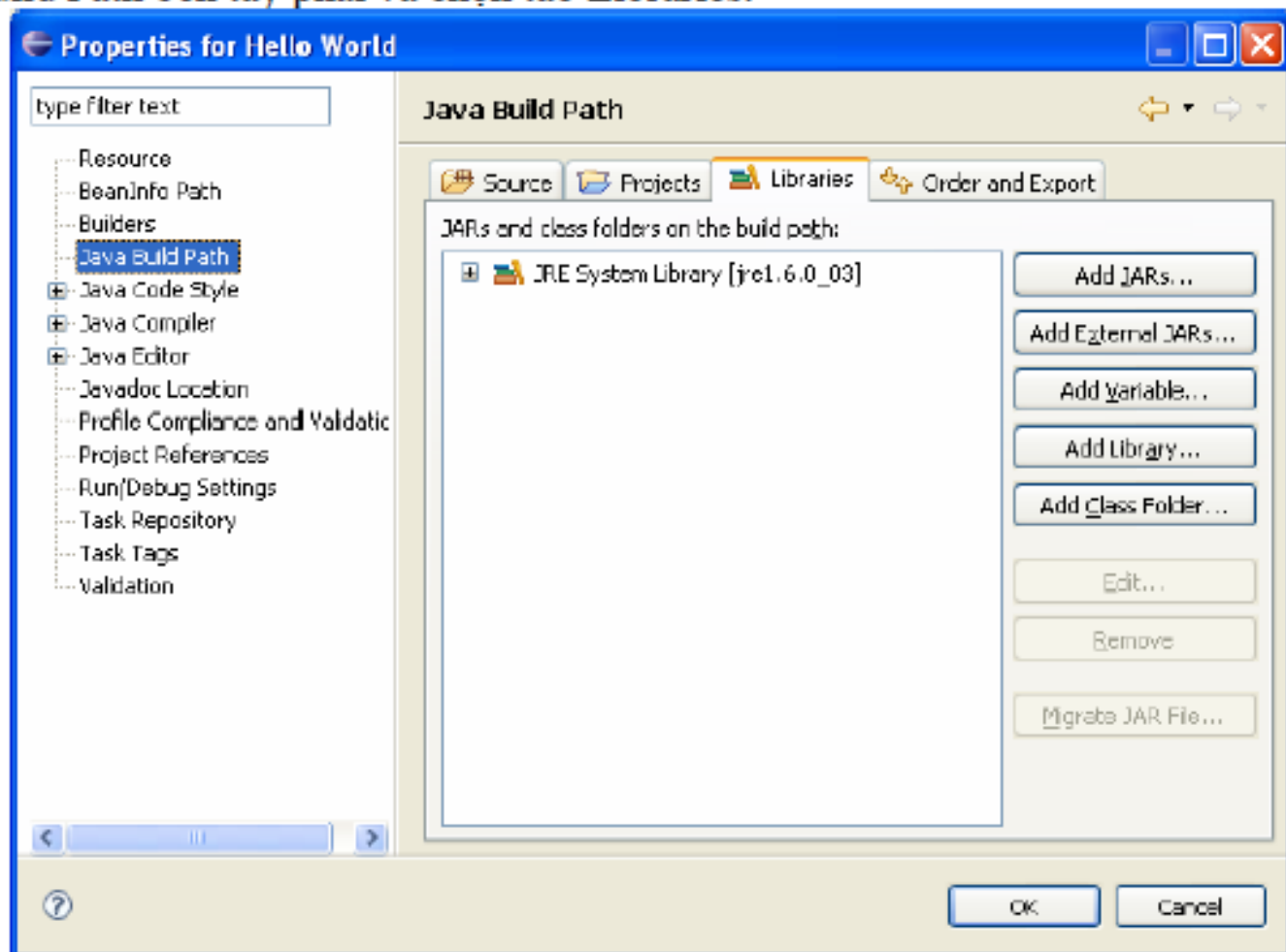


CÔNG CỤ JUNIT

4. Cấu hình hệ thống

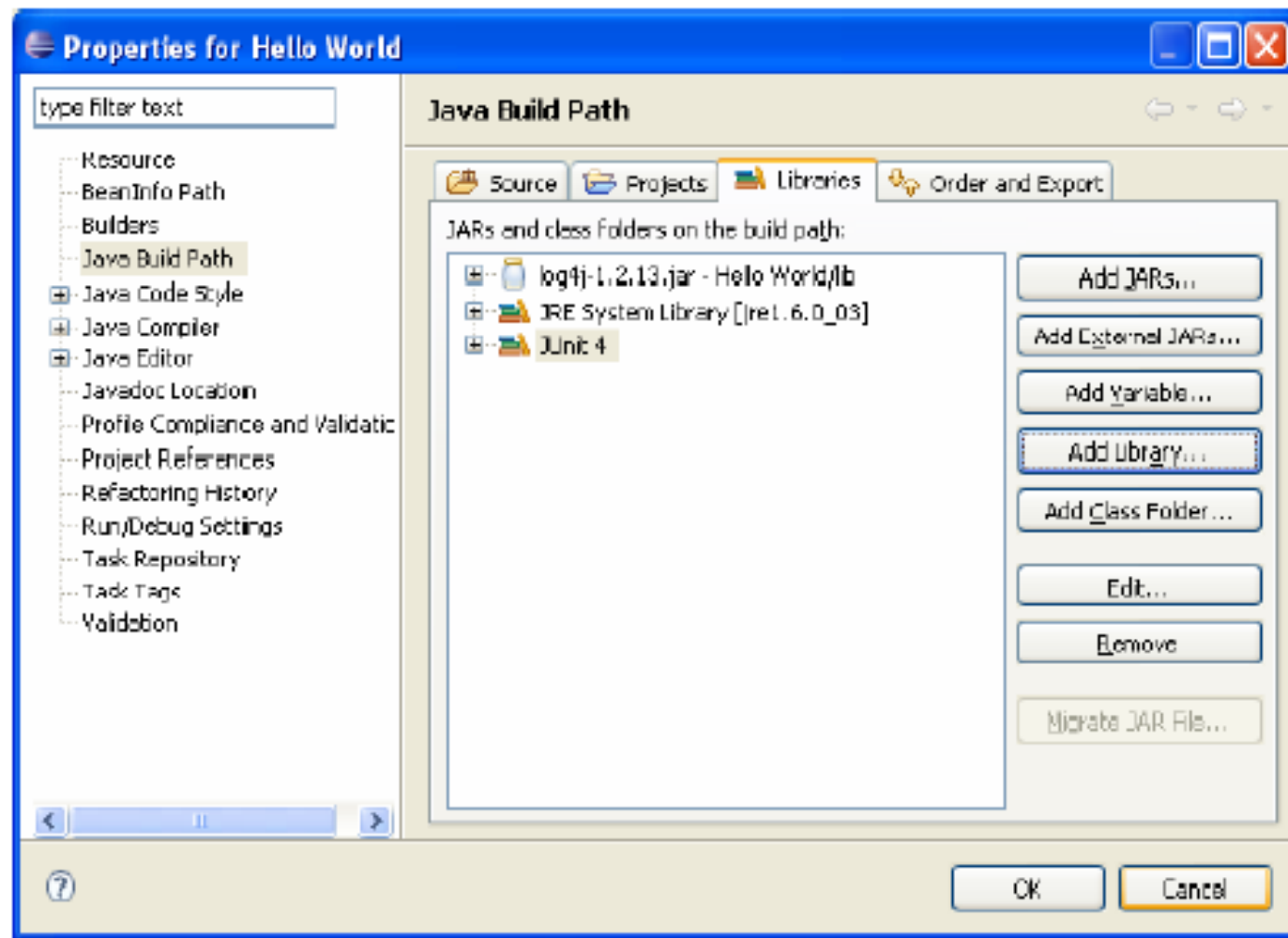
a. Thêm các thư viện vào project

Các thư viện trong java thường được lưu thành các file jar. Để sử dụng các thư viện này trong project, bạn hãy chọn Project → Properties. Trong cửa sổ mới hiện ra, chọn mục Java Build Path bên tay phải và chọn tab Libraries.



CÔNG CỤ JUNIT

Tiếp đến ta sẽ thêm thư viện JUnit. Thư viện JUnit cũng là mã nguồn mở, chuyên dùng cho việc thiết kế và chạy các test case. Thư viện này đã có sẵn trong Eclipse nên bạn chỉ cần chọn Add Library → JUnit. Chọn JUnit 4 và Finish. Màn hình quản lý project bây giờ sẽ giống như sau:

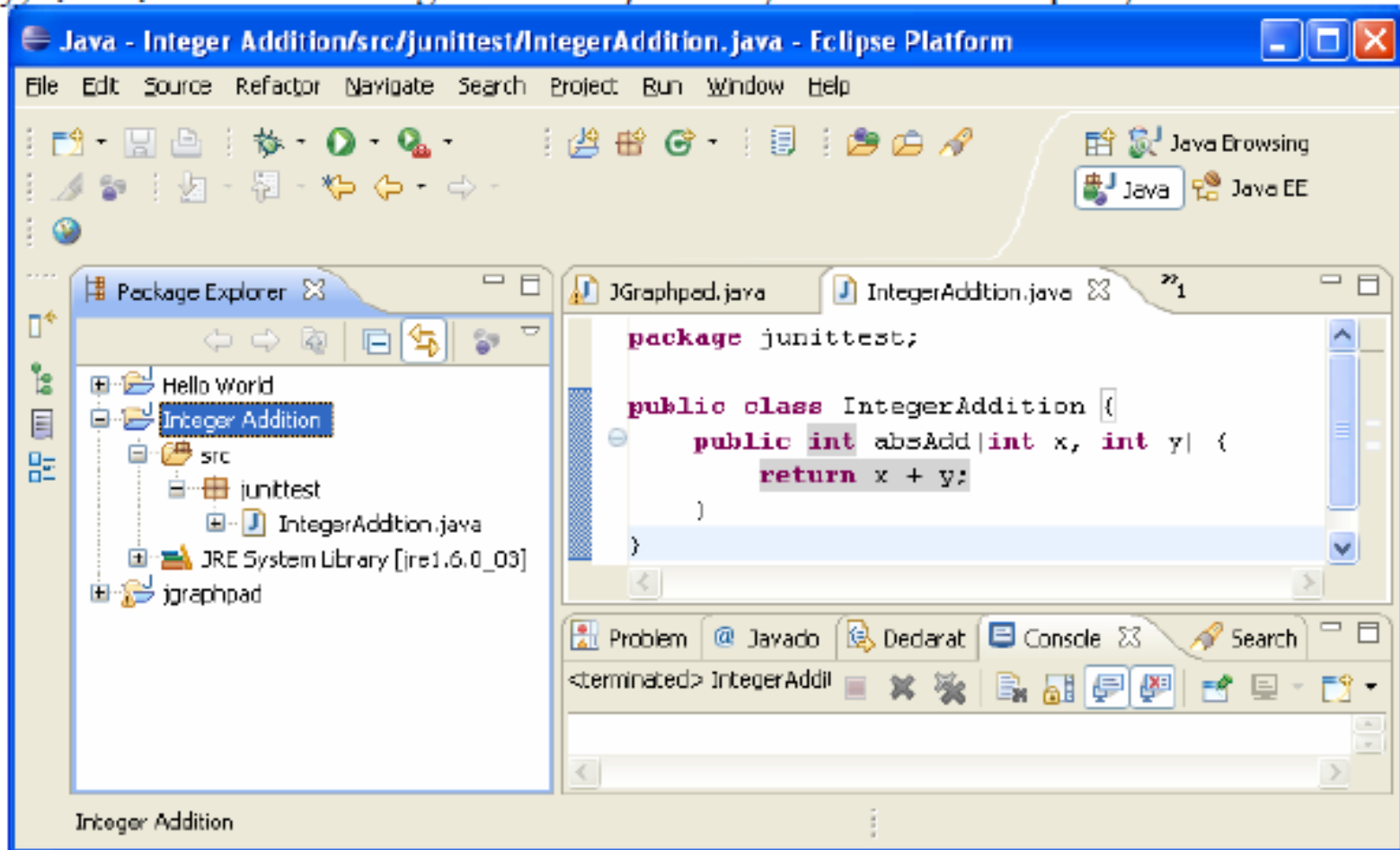


CÔNG CỤ JUNIT

10. Chức năng testing với JUnit

JUnit là một gói phần mềm mã nguồn mở đi kèm với Eclipse, chuyên dùng cho việc thiết kế và chạy thử các test case. Ví dụ này sẽ cho bạn một số khái niệm về JUnit. Bạn có thể tìm hiểu thêm về JUnit tại trang web www.junit.org

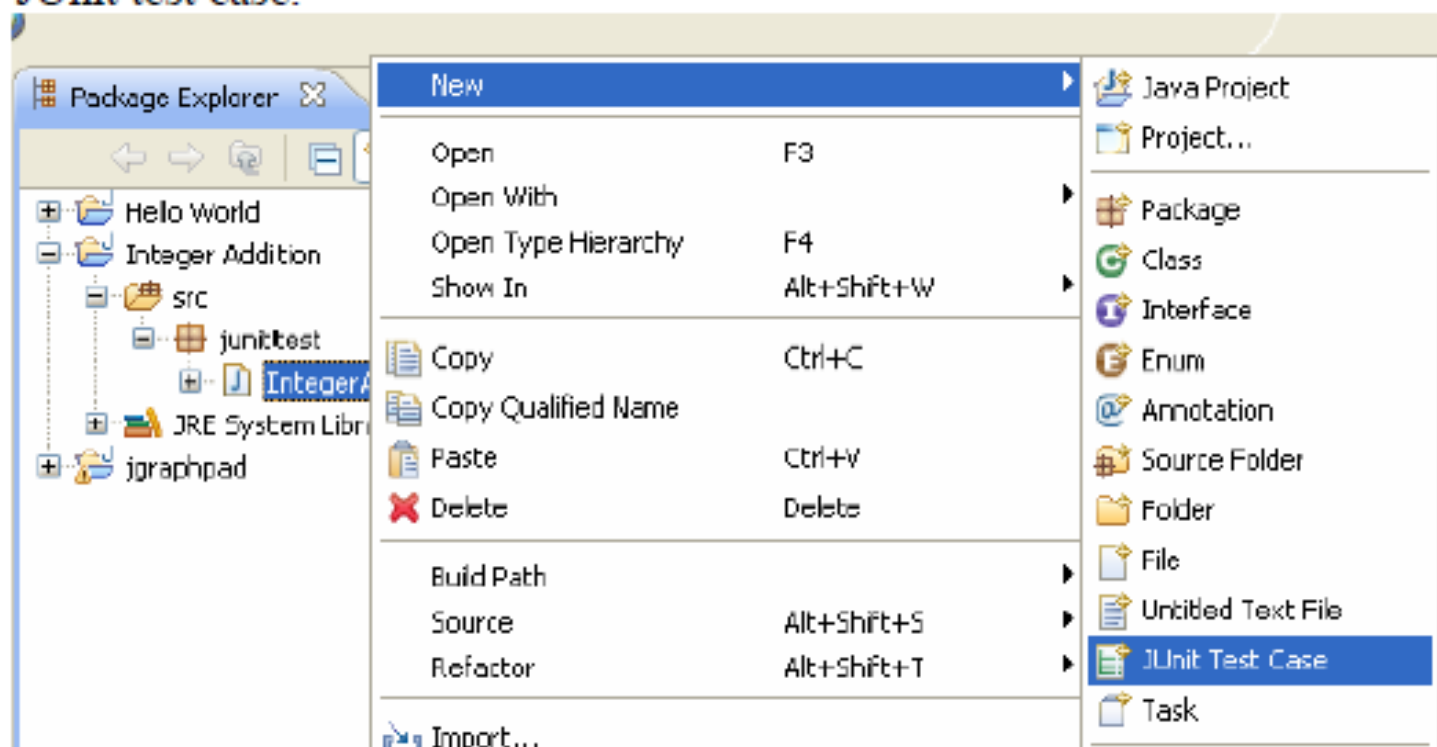
Chúng ta hãy tạo một project mới trong Eclipse, lấy tên là Integer Addition. Trong project này, tạo một file mới là IntegerAddition.java. Hãy viết code cho lớp này như hình sau:



CÔNG CỤ JUNIT

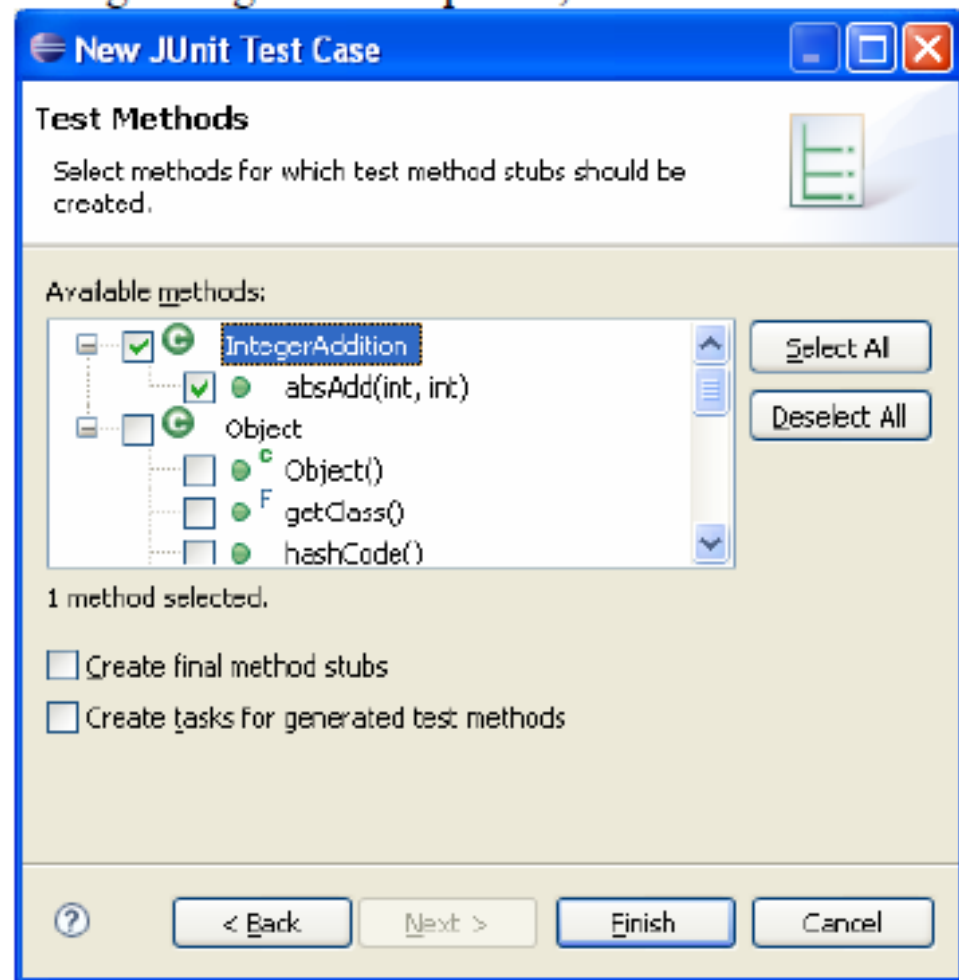
Trong lớp này, ta có một phương thức *absAdd*, mục đích của nó là cộng giá trị tuyệt đối của 2 số nguyên. Như bạn thấy, chúng ta đã cố tình hiện thực sai phương thức này để thử nghiệm tính năng của JUnit.

Hãy tạo một JUnit test case bằng cách click chuột phải vào file IntegerAddition, chọn New → JUnit test case.



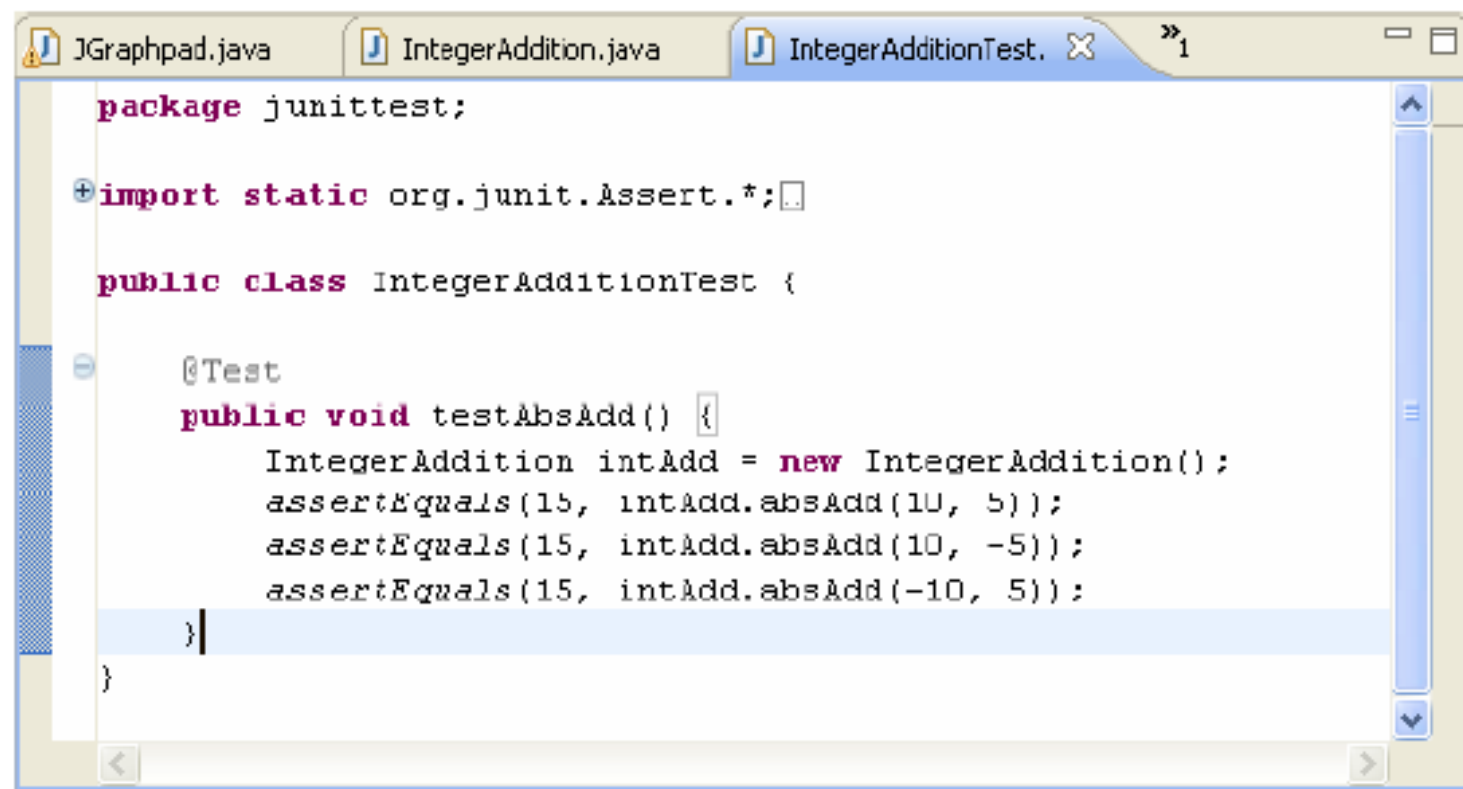
CÔNG CỤ JUNIT

Trong khung cửa sổ tiếp theo, check vào ô `absAdd` như hình dưới:



Sau đó OK, bạn sẽ có một lớp mới tên là `IntegerAdditionTest`. Lớp này đã có sẵn phương thức `testAbsAdd`. Bạn có thể viết thêm các phương thức khác, nhưng chỉ những phương thức có chỉ thị `@Test` ở đầu mới được JUnit test thử. Bạn hãy hiện thức các phương thức này như sau:

CÔNG CỤ JUNIT

A screenshot of a Java IDE window showing three tabs: JGraphpad.java, IntegerAddition.java, and IntegerAdditionTest.java. The IntegerAdditionTest.java tab is active, displaying the following code:

```
package junittest;

import static org.junit.Assert.*;

public class IntegerAdditionTest {

    @Test
    public void testAbsAdd() {
        IntegerAddition intAdd = new IntegerAddition();
        assertEquals(15, intAdd.absAdd(10, 5));
        assertEquals(15, intAdd.absAdd(10, -5));
        assertEquals(15, intAdd.absAdd(-10, 5));
    }
}
```

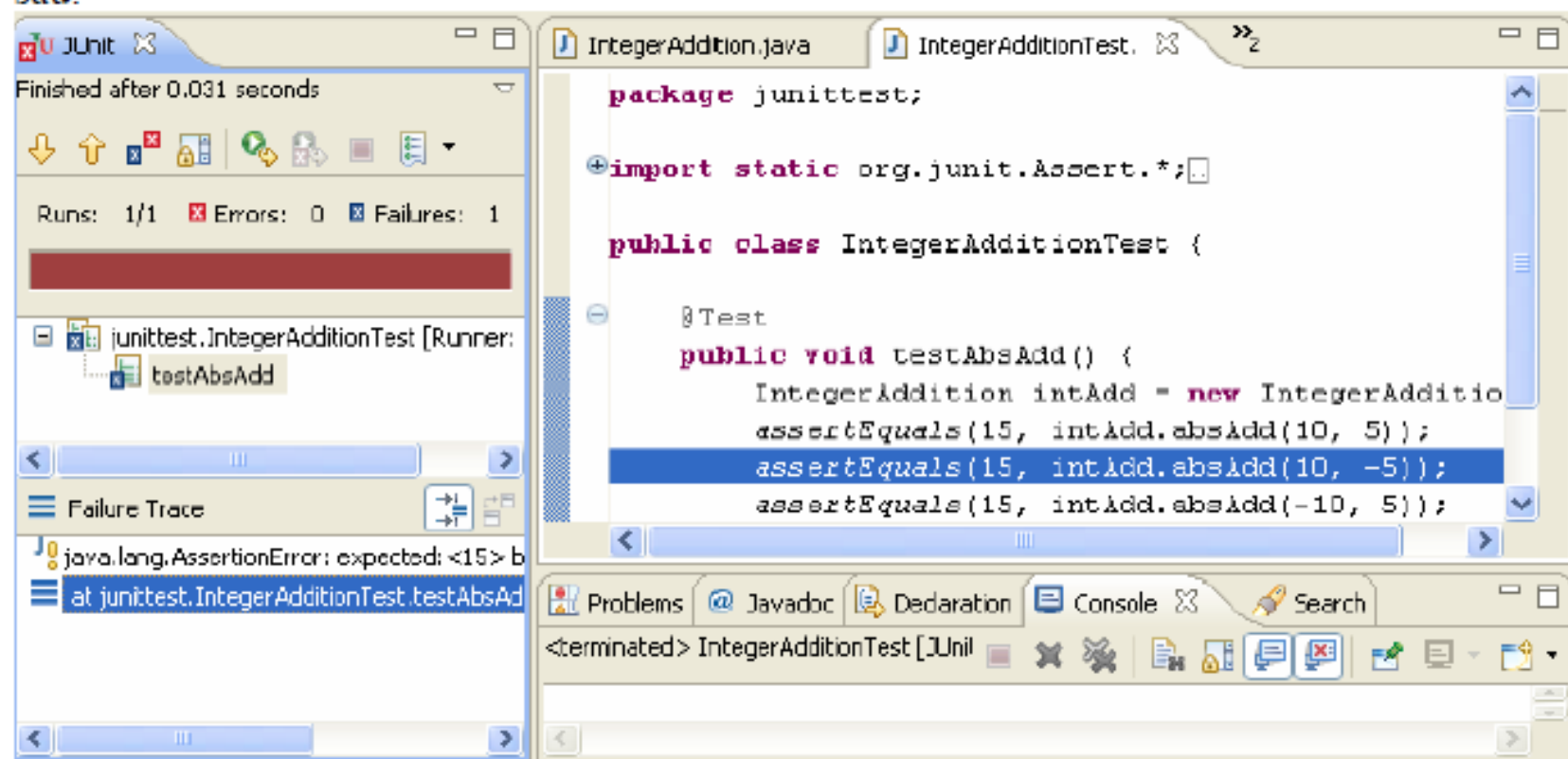
Ý nghĩa của đoạn code trên là như sau: đầu tiên ta khai báo một biến *intAdd* thuộc lớp *IntegerAddition* và gọi constructor của lớp này. Tiếp theo là 3 hàm *assertEquals* dùng để test cho 3 trường hợp.

Các hàm *assertEquals* cho thấy các “kỳ vọng” về kết quả đạt được của chương trình. Ý nghĩa của 3 dòng test trên là: ta hy vọng hàm *absAdd* sẽ trả về giá trị 15 khi nhận được input là 10 và 5; và tương tự, hy vọng hàm này sẽ trả về giá trị 15 khi nhận các cặp input lần lượt là 10 và -5, -10 và 5.

CÔNG CỤ JUNIT

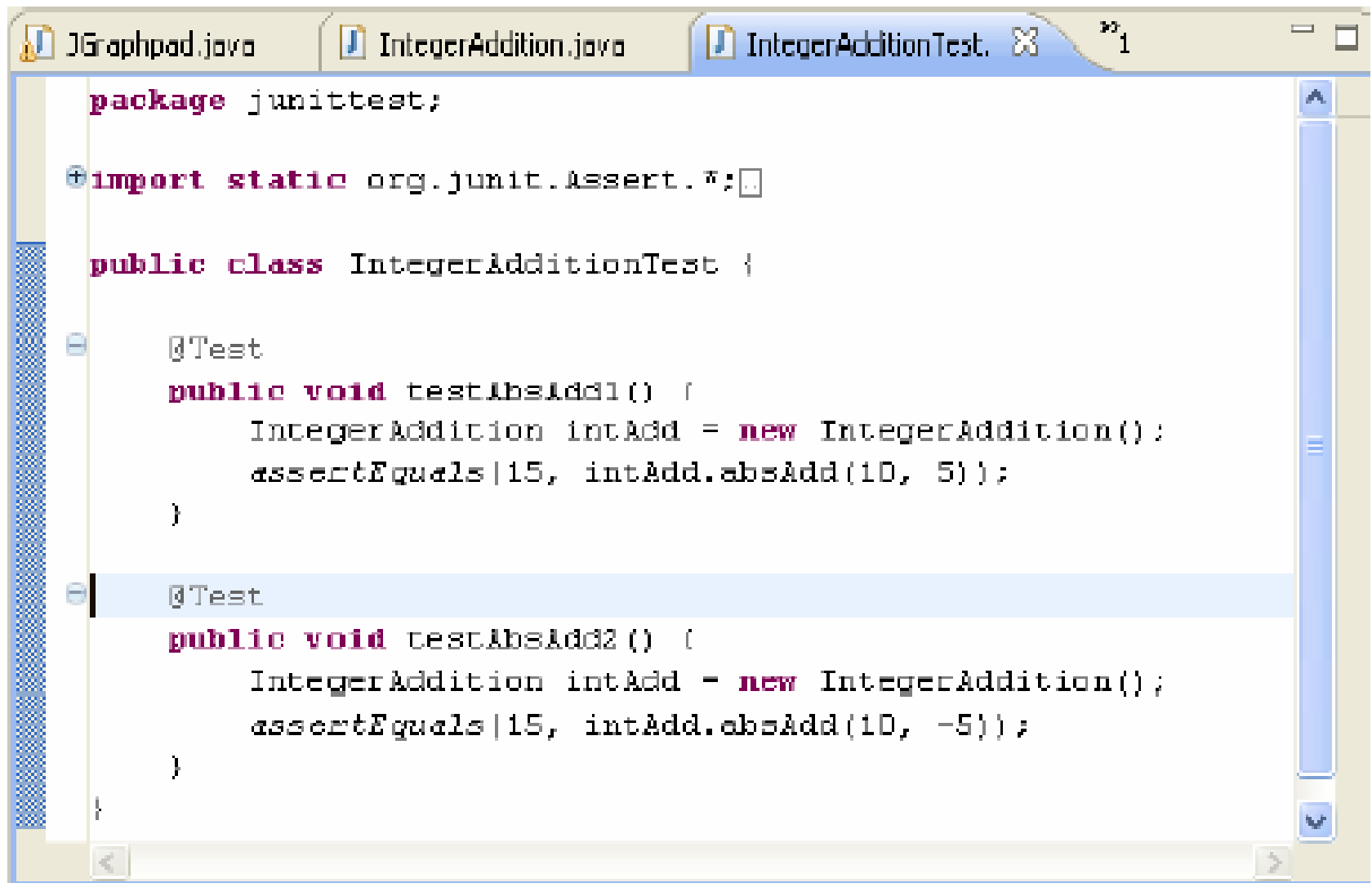
Mỗi khi gặp hàm *assertEquals*, JUnit sẽ kiểm tra xem hai thông số được đưa vào có bằng nhau không. Nếu bằng, chương trình sẽ chạy tiếp; nếu không, JUnit sẽ dừng phương thức được test lại và báo về một failure. Một phương thức bị failure sẽ không ảnh hưởng gì đến các phương thức cần test khác.

Bây giờ hãy chạy thử test case này. Hãy chọn Run → Run As → JUnit Test. Kết quả như sau:



JUnit cho ta biết có 1 test case bị hỏng, đồng thời nếu bạn double click vào dòng trong khung Failure Trace ở dưới, nó sẽ cho bạn biết chương trình bị fail ở test case nào. Bây giờ hãy thử hiện thực lại lớp IntegerAdditionTest theo hình dưới đây

CÔNG CỤ JUNIT



```
package junittest;

import static org.junit.Assert.*;

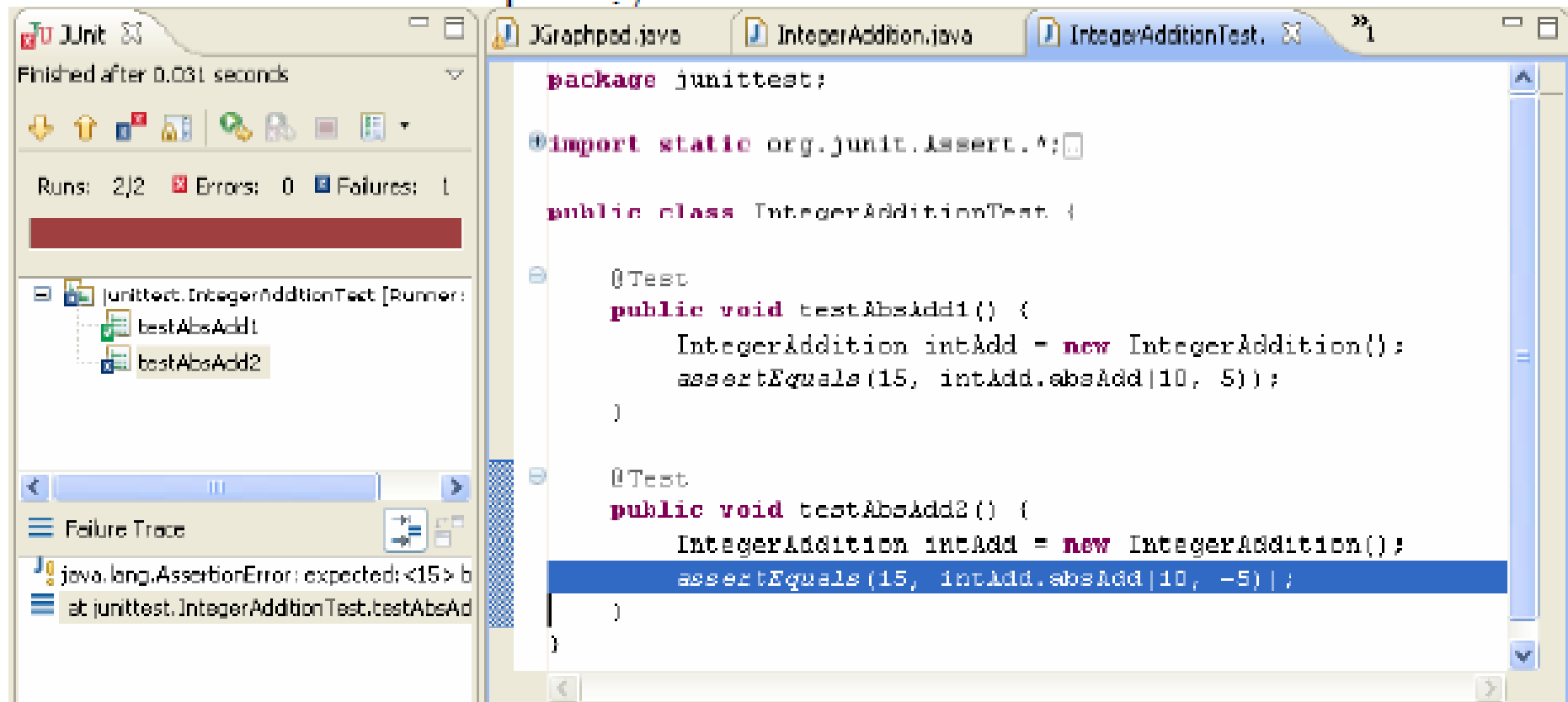
public class IntegerAdditionTest {

    @Test
    public void testAbsAdd1() {
        IntegerAddition intAdd = new IntegerAddition();
        assertEquals(15, intAdd.absAdd(10, 5));
    }

    @Test
    public void testAbsAdd2() {
        IntegerAddition intAdd = new IntegerAddition();
        assertEquals(15, intAdd.absAdd(10, -5));
    }
}
```

CÔNG CỤ JUNIT

Ta đã có 2 test case. Kết quả chạy thử như sau:



Như bạn thấy, JUnit thông báo đã chạy được 2 test case và có một failure.

Junit(4)
- within/under
NetBeans -

JUnit within/under NetBeans

Practical Manual:

Practical conventions

How to build unit tests?

How to run unit tests?

Demo task

Practical exercise task

Practical conventions

Each production code class `X` has a test code class `Xtest` derived from `TestCase` class

Each method `y(){...}` from class `X` has a method `testy(){...}` from `Xtest` class

Each test code class `Xtest` has `setUp()` method and `tearDown()` method

Practical conventions

All production code located in a separate project folder Source Packages

All test code located in another separate project folder Test Packages

The skeletal structure and mostly the contents of a test code class Xtest and test code methods generated automatically

How to build unit tests?

Part 1

Create New project

Add Java class with main() method

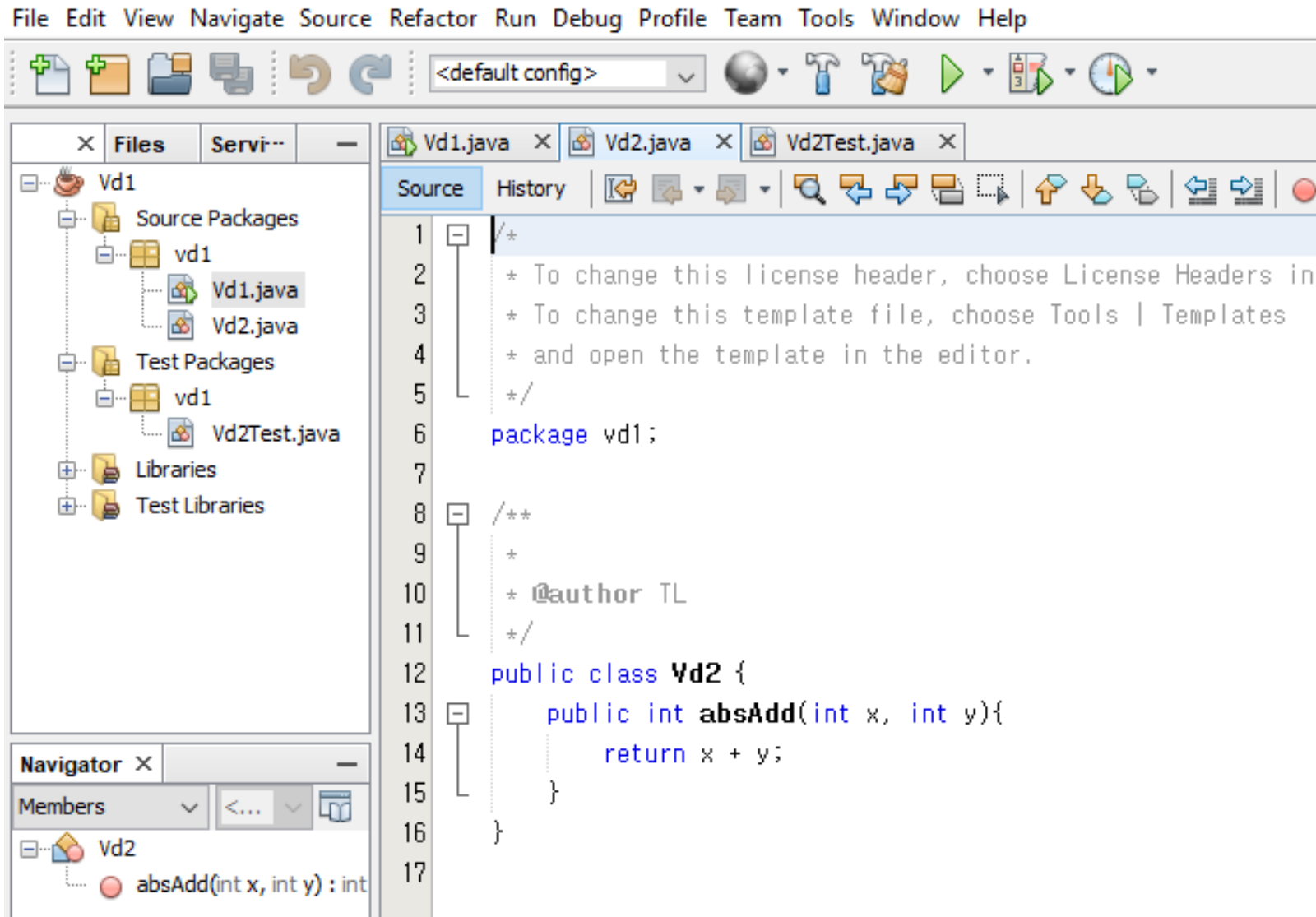
Add user defined class to be tested
as a separate file within the same
package, or

as part of file with Java main() class

Build the project and run it as a regular
Java application

Run > Run Main project (F6)

How to run unit tests?



How to run unit tests?

Part 2a

Select Java class to be tested (from Projects...)

Click the right mouse button and invoke
Tools > Create Tests, **select JUNIT4**

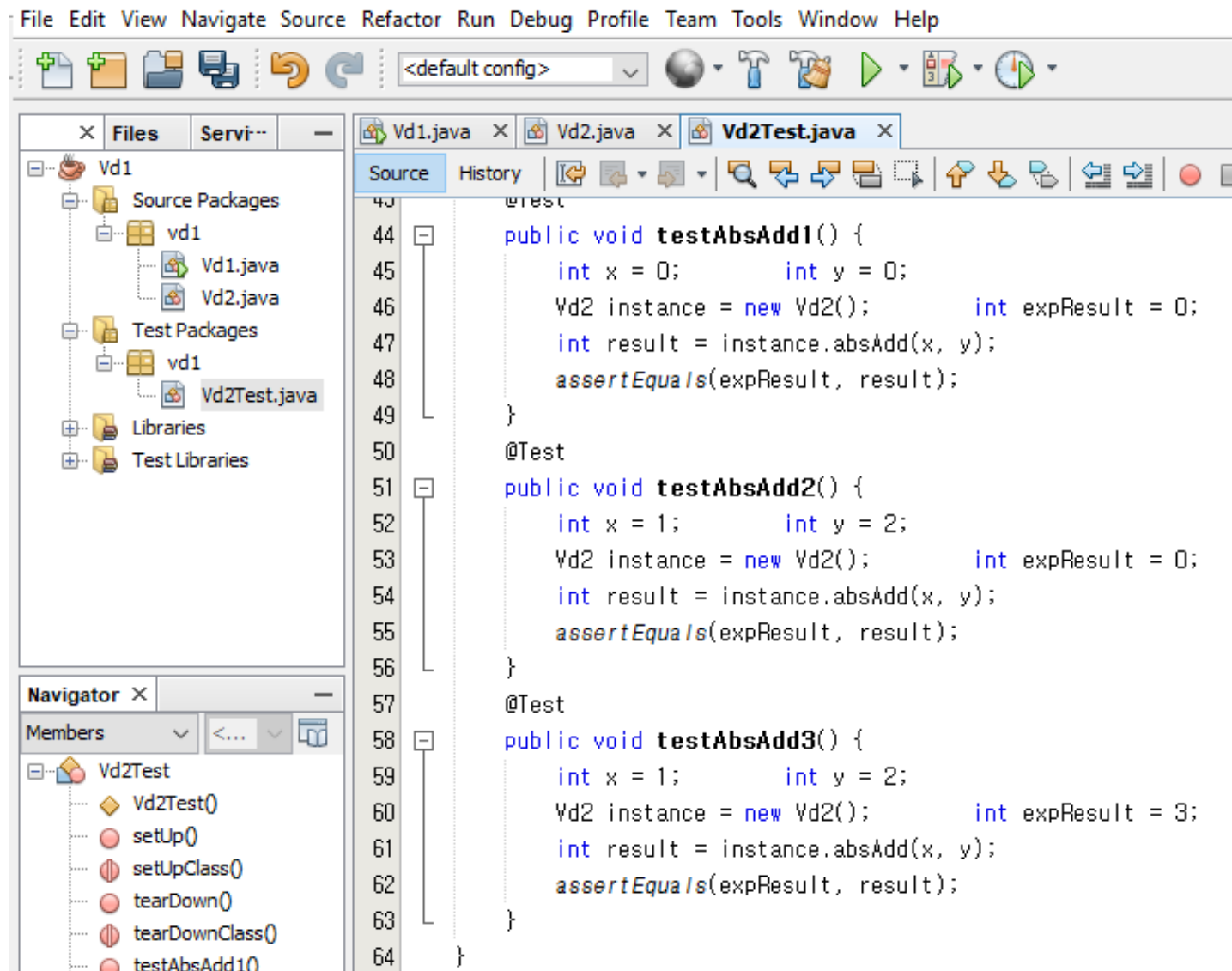
Modify the generated text if necessary

Re-Build the project and run it as a JUnit test

Run > Test Project(...) (Alt+F6)

Analyze the test results

How to run unit tests?



How to run unit tests?

The screenshot displays an IDE interface for running unit tests. On the left, a project tree lists various projects, with 'swtesting' highlighted. The middle pane shows the source code for 'swtesting.Vd2Test', which contains three test methods: `testAbsAdd1()`, `testAbsAdd2()`, and `testAbsAdd3()`. Each method tests the `absAdd` method of the `Vd2` class with different input values. The right pane shows the test results, indicating that 2 tests passed and 1 test failed. The failed test is `testAbsAdd2`, which failed because the expected result was 3 but the actual result was -1.

```
32  @Test
33  public void testAbsAdd1() {
34      int x = 0;  int y = 0;  int expResult = 0;
35      Vd2 instance = new Vd2();
36      int result = instance.absAdd(x, y);
37      assertEquals(expResult, result);
38  }
39  @Test
40  public void testAbsAdd2() {
41      int x = 1;  int y = -2;  int expResult = 3;
42      Vd2 instance = new Vd2();
43      int result = instance.absAdd(x, y);
44      assertEquals(expResult, result);
45  }
46  @Test
47  public void testAbsAdd3() {
48      int x = 1;  int y = 2;  int expResult = 3;
49      Vd2 instance = new Vd2();
50      int result = instance.absAdd(x, y);
51      assertEquals(expResult, result);
52  }
```

tearDown - Navigator x

Members

- setUp()
- setUpClass()
- tearDown()
- tearDownClass()
- testAbsAdd1()
- testAbsAdd2()
- testAbsAdd3()

Test Results x

swtesting.Vd2Test x

Tests passed: 66.67 %

2 tests passed, 1 test failed. (0.958 s)

- swtesting.Vd2Test Failed
 - testAbsAdd1 passed (0.009 s)
 - testAbsAdd2 Failed: expected:<3> but was:<-1>
 - testAbsAdd3 passed (0.001 s)

How to run unit tests?

OR
(follow option 2b)

How to run unit tests?

Part 2b

Add a new File to the project

File > new File...

Select category: Unit Test

Select File Type: Test for Existing Class

You have to specify the class to be tested

You have to select **JUnit version: 4**

The test class automatically generated

Modify the generated text if necessary

Re-Build the project and run it as a JUnit test

Run > Test Project(...) (Alt+F6)

Analyze the test results

Asserting expectations

`assertEquals(expected, actual)`

`assertEquals(message, expected, actual)`

`assertEquals(expected, actual, delta)`

`assertEquals(message, expected, actual, delta)`

`assertFalse((message), condition)`

`assertTrue((message), condition)`

`Assert(Not)Null(object)`

`Assert(Not)Null(message, object)`

`Assert(Not)Same(expected, actual)`

`Assert(Not)Same(message, expected, actual)`



Copyright 2005 Hans Bjordahl

Bug Bash by Hans Bjordahl

<http://www.bugbash.net/>



Copyright 2005 Hans Bjordahl

Bug Bash by Hans Bjordahl

<http://www.bugbash.net/>



Bug Bash by Hans Bjordahl



Copyright 2005 Hans Bjordahl

<http://www.bugbash.net/>



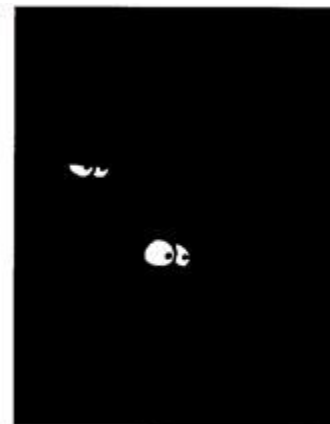
Bug Bash by Hans Bjordahl



Copyright 2005 Hans Bjordahl



<http://www.bugbash.net/>



Copyright 2005 Hans Bjordahl

Are you sure you're sure?



Bug Bash by Hans Bjordahl



<http://www.bugbash.net/>

Đồ án môn học

Mỗi nhóm sinh viên từ 2-4 người chọn một đề tài để nghiên cứu và báo cáo:

Đề tài 1: Công cụ hỗ trợ kiểm thử tự động JUnit

Đề tài 2: Công cụ kiểm thử tự động Selenium

Đề tài 3: Công cụ hỗ trợ kiểm thử tự động AutoIT

Đề tài 4: Hệ thống quản lý bug Bugzilla

Đề tài 5: Kiểm thử trên thiết bị di động (mobile testing)

Đề tài 6: Công cụ hỗ trợ kiểm thử tự động Robotium

Đề tài 7: Tự chọn, có sự đồng ý của GVHD